between MATLAB (on the left side of the figure) and GNU Octave (on the right side of the figure). The first difference shown in Figure 4 is due to the different ways in which character strings and character arrays are treated by the two languages. The second difference is due to GNU Octave supporting the post-increment operator available in programming languages like C, C++, Java, etc. From Figure 4, it is clear that MATLAB does not support this operator, whereas its behaviour is similar in C, C++, Java, and GNU Octave. Keep in mind that these are not the only syntactical differences between MATLAB and GNU Octave. As the differences can be more subtle, GNU Octave users should be careful when using MATLAB code.

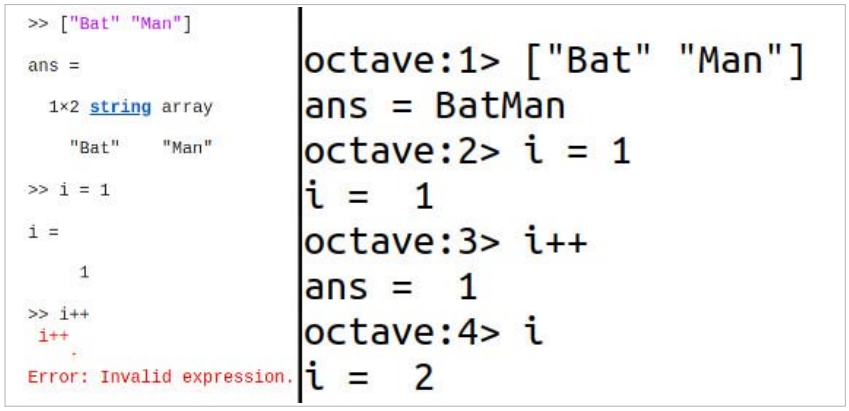

Figure 4: Syntactic differences between MATLAB and GNU Octave

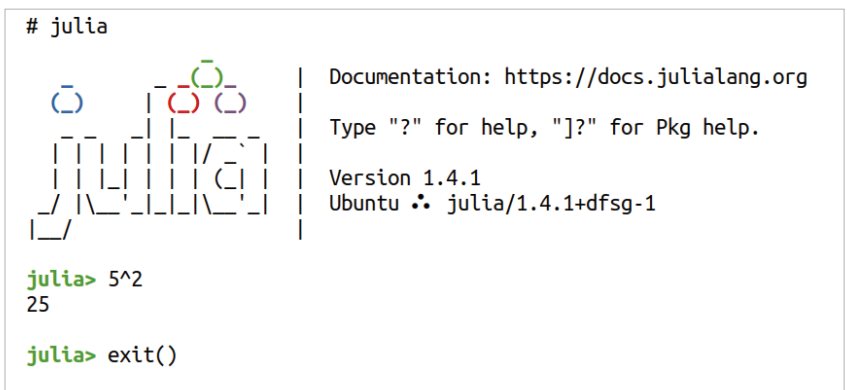

Figure 5: Julia REPL

Introduction to Julia

Now, it is time to introduce Julia, a programming language that might pose the greatest threat to the dominance of Python in the development of AI and machine learning-based applications. Julia is a relatively new, high-level programming language designed for computational science. It was first released in 2012, and its features make it well-suited for developing AI and machine learning-based applications. Python and Julia can be compared using the analogy of a mobile phone camera and a digital camera. Python, being a widely-used programming language, is as versatile as a mobile phone and can be utilised for various applications, including AI-based development. In contrast, Julia is like a digital camera specialised in a particular task. Julia is designed for high-performance computing and numerical analysis. It is a relatively new language that has gained prominence in the scientific computing community due to its speed and efficiency. Julia's syntax is somewhat similar to Python's, but it is optimised for speed and efficiency, making it ideal for complex numerical computations and simulations. Thus, speed is one of the important reasons why you should prefer Julia over Python.

As Julia shows great potential as a programming language for AI in the future, let us delve into it further. To install Julia on Ubuntu, simply enter the command '*sudo apt-get install julia*' in your terminal. Then use the command '*julia*' to launch the Julia REPL (Read-Eval-Print Loop), a command-line interface for working with Julia. To exit the Julia REPL, simply type '*exit( )*' and press *Enter.* In Figure 5, you can see an example of the Julia REPL running a mathematical expression (5^2 = 25).

Now, let us write and execute a Julia program. Notice that Julia programs have a '*.jl*' extension. Consider the Julia program called '*pgm1.jl*' shown below, which generates a random array of integers, and calculates its mean and standard deviation.

```
using Statistics
Arr = rand(1:100, 10)
μ = mean(Arr)
σ = std(Arr)
println("Array: $Arr")
println("Mean: $μ")
println("Standard Deviation: $σ")
```

The line of code '*using Statistics*' loads the package *Statistics*, which provides several statistical functions, including mean and standard deviation. The line of code '*Arr = rand(1:100, 10)*' creates a one-dimensional array of length 10, filled with random integers between 1 and 100 (inclusive). The rest of the code calculates the mean and standard deviation of the array *Arr*, and prints the array *Arr* and the calculated results. Julia is a programming language intended for scientific computing and allows the use of mathematical symbols as variable